Journal of King Saud University Computer and Information Sciences (2026) 38:780
https://doi.org/10.1007/s44443-026-01193-5

RESEARCH

Topo-MFP: Topology-aware multimodal propagation learning for fault detection and root cause localization in microservice systems

Peng Zhang[1] · Junsheng Wu[1] · Hao He[1] · Longfei Yan[1] · Weigang Li[1]

Received: 12 June 2026 / Accepted: 1 August 2026
© The Author(s) 2026

Abstract

Fault detection and root cause localization in microservice systems are challenging because runtime failures may propagate across dependent services and appear heterogeneously in metrics, logs, and traces. Existing approaches often rely on single-modality observations, local anomaly scores, or static service topology, which may be insufficient to distinguish root-cause services from downstream affected services. To address this problem, this paper proposes Topo-MFP, a topology-aware multimodal propagation learning framework for service-level fault detection and root cause localization. Topo-MFP integrates heterogeneous observability data with service dependency information in two stages. For fault detection, it constructs a semantic-transition log template graph by combining log template semantics with transition statistics, and learns topology-aware service representations from dynamic service dependency graphs through spatio-temporal graph attention. For root cause localization, it learns diagnosis-oriented multimodal propagation graphs from metrics, logs, and traces. The modality-specific propagation graphs are fused using cross-modal consistency and a topology-aware soft sparsity prior, where different penalties are assigned to topology-adjacent and topology-distant service pairs. Root-cause candidates are then ranked by symptom-to-source reverse random walk with restart over the learned propagation graph. Experiments are conducted on AIOps2022 and a TrainTicket-based TT dataset. Over five random seeds, Topo-MFP achieves mean F1-scores of 0.949 and 0.951 for fault detection, and mean Hit@1 scores of 0.872 and 0.901 for root cause localization on AIOps2022 and TT, respectively. Comparisons with Random, Local-Score, and Topology-RWR baselines show that the learned multimodal propagation graph provides diagnostic information beyond random ranking, local anomaly scores, and raw invocation topology. Ablation and sensitivity analyses further indicate the contributions of semantic log modeling, cross-modal consistency, and topology-aware regularization. The learned propagation graph is used as a diagnosis-oriented ranking representation rather than as a verified causal graph.

Keywords Microservice systems · Fault detection · Root cause localization · Multimodal learning · Graph neural networks · Service dependency topology

1 Introduction

Microservice systems have become a widely adopted paradigm for building cloud-native applications. By decomposing an application into independently developed, deployed, and scaled services, microservices improve maintainability, scalability, and deployment flexibility (Di Francesco et al. 2017; Baškarada et al. 2020; Lewis and Fowler 2014; Dragoni et al. 2017). However, the same decomposition also introduces complex service dependencies and dynamic runtime behaviors. A local fault in one service may propagate through invocation relationships and trigger abnormal symptoms in multiple downstream or upstream services. Therefore, accurate fault detection and root cause localization

✉ Peng Zhang
zhangpengjason@mail.nwpu.edu.cn

✉ Junsheng Wu
wujunsheng@nwpu.edu.cn

✉ Weigang Li
liweigang@nwpu.edu.cn

Hao He
352384669@qq.com

Longfei Yan
18509247217@163.com

[1] School of Software, Northwestern Polytechnical University, Xi'an, China

Published online: 25 August 2026

are essential for maintaining the reliability of microservice systems.

Modern microservice systems continuously generate heterogeneous observability data, including metrics, logs, and traces (Cinque et al. 2016; Sigelman et al. 2010). Metrics describe numerical runtime states such as CPU usage, memory usage, response latency, throughput, and error rate. Logs record execution events, warnings, exceptions, and business-level messages. Traces reveal request-level invocation paths and span-level latency across distributed services. These data sources provide complementary diagnostic evidence. However, they differ significantly in format, temporal granularity, and semantic meaning, making effective multimodal learning a challenging task.

Although many studies have investigated fault diagnosis in microservice systems, several limitations remain. First, many log-based methods treat log templates as symbolic event identifiers and do not sufficiently exploit the semantic information contained in log texts. For example, "connection timeout" and "database connection failed" may have different lexical forms but similar fault semantics, while "user login success" and "user login failed" may be lexically similar but have opposite operational meanings. Second, many multimodal methods combine metrics, logs, and traces through simple concatenation or weighted aggregation, which may fail to capture cross-modal interactions and modality-specific diagnostic evidence. Third, service dependencies and temporal symptom propagation are often insufficiently modeled, making it difficult to distinguish root-cause services from downstream affected services that only exhibit propagated symptoms.

To address these issues, this paper proposes Topo-MFP, a topology-aware multimodal propagation learning framework for microservice fault detection and root cause localization. Topo-MFP jointly models log semantics, metric dynamics, trace-derived service dependencies, and temporal symptom propagation. Instead of relying on a single type of monitoring data, it learns service-level diagnostic representations from heterogeneous observability data and performs symptom-to-source ranking over a learned diagnosis-oriented propagation graph. The learned graph is used as a ranking-oriented diagnostic representation, rather than as a verified causal propagation graph.

The main contributions of this paper are summarized as follows.

1) We formulate service-level root cause localization as symptom-to-source ranking over a learned diagnosis-oriented propagation graph. Instead of applying random walk directly to the raw invocation topology, Topo-MFP performs reverse random walk with restart over a multimodal propagation graph learned from temporal evidence, abnormal observations, and topology priors.
2) We introduce a topology-aware soft sparsity prior for propagation graph learning. The proposed prior uses different penalties for topology-adjacent and topology-distant service pairs, denoted by ρ_in and ρ_out, respectively. This design encourages the learned propagation graph to respect observed invocation dependencies while still allowing implicit dependencies when supported by multimodal evidence.
3) We incorporate semantic-transition log modeling into service-level diagnosis by representing log templates with Sentence-BERT-based semantic attributes and constructing directed template-transition edges from training log sequences. This component provides semantic and execution-order evidence for downstream fault detection and RCA.
4) We build a topology-aware multimodal representation module for service-level fault detection by aligning metrics, logs, and traces at the service-window level and incorporating trace-derived dynamic service dependencies through spatio-temporal attention.
5) We evaluate Topo-MFP on AIOps2022 and a Train-Ticket-based TT dataset under a unified service-level protocol. Experimental results, ablation studies, and controlled RCA baselines demonstrate the effectiveness of semantic log modeling, topology-aware representation learning, multimodal propagation graph fusion, and symptom-to-source ranking.

Overall, Topo-MFP focuses on service-level diagnosis by combining topology-aware multimodal representation learning with propagation-oriented symptom-to-source ranking. The framework is designed to improve operational diagnosis rather than to provide intervention-level causal identification.

2 Related work

2.1 Observability data in microservice systems

Observability data provide the foundation for monitoring and diagnosis in microservice systems. In modern cloud-native applications, the three major observability sources are metrics, logs, and traces (Cinque et al. 2016). Metrics characterize numerical runtime states such as CPU usage, memory usage, latency, throughput, and error rate; logs record textual execution events, warnings, exceptions, and business-level states; and traces capture request-level invocation paths and span-level latency across distributed services. Dapper is one of the earliest large-scale distributed tracing systems and has inspired many later tracing tools (Sigelman et al. 2010).

Although these data sources provide complementary diagnostic evidence, they differ substantially in format, temporal granularity, and semantic meaning. This heterogeneity makes it difficult to align multimodal observations and infer service-level abnormal behavior in complex microservice systems.

2.2 Learning-based fault detection

Log-based anomaly detection methods model normal execution patterns from logs. OASIS constructs control-flow graphs from execution logs for anomaly detection (Nandi et al. 2016). LogSed builds time-weighted control-flow graphs to detect sequence and latency anomalies (Jia et al. 2017). DeepLog, LogAnomaly, and LogRobust use deep learning models to learn log sequence patterns (Du et al. 2017; Meng et al. 2019; Zhang et al. 2019). However, many log-based methods still treat log templates as symbolic events or use semantic representations without integrating service topology.

Trace-based methods analyze invocation paths, response time, and structural patterns. Seer combines convolutional and recurrent neural networks to predict QoS violations from traces and queueing information (Gan et al. 2019). TraceAnomaly uses service-level deep Bayesian networks for unsupervised trace anomaly detection (Liu et al. 2020). Other studies detect anomalies by comparing execution traces or learning self-supervised representations from distributed traces (Nedelkoski et al. 2019; Bogatinovski et al. 2020; Meng et al. 2021).

Metric-based methods model multivariate time series through statistical models, Markov models, autoencoders, or graph neural networks. USAD is an unsupervised autoencoder-based method for multivariate time-series anomaly detection (Audibert et al. 2020). Recent multimodal methods, such as UAC-AD and TraceVAE, demonstrate the value of multimodal and graph-based modeling (Liu et al. 2024; Xie et al. 2023). However, effectively aligning log semantics, metric dynamics, and trace topology remains challenging.

2.3 Root cause localization in microservice systems

Root cause localization aims to identify the service or component that triggers abnormal symptoms. MicroRCA uses performance metrics and dependency graphs to locate performance issues (Wu et al. 2020). MicroHECL constructs dynamic service dependency graphs and analyzes abnormal propagation paths (Liu et al. 2021). Grace introduces graph convolutional networks for interpretable root cause analysis in microservices (Ren et al. 2023). MicroEGRCL learns edge-aware attention over service invocation graphs (Chen et al. 2022). RUN applies neural Granger causal discovery for microservice RCA (Lin et al. 2024). DGERCL learns dynamic graph embeddings for RCA (Cheng et al. 2024). DiagFusion, MMRCA, MRCA, and CHASE integrate multimodal observability data for failure diagnosis and root cause analysis (Zhang et al. 2023; White et al. 2021; Wang et al. 2024; Zhao et al. 2024).

Different from strict structural causal discovery, Topo-MFP learns a diagnosis-oriented propagation dependency graph. The learned graph is used for root cause ranking rather than intervention-level causal explanation.

2.4 Comparison with representative methods

Table 1 summarizes the technical positioning of Topo-MFP.

The comparison focuses on key modeling mechanisms, including log semantic modeling, topology modeling, temporal modeling, multimodal fusion, and RCA strategy.

Table 1 Technical comparison between Topo-MFP and representative microservice diagnosis methods

Method	Log semantics	Dynamic topology	Temporal modeling	Multimodal fusion	RCA strategy
USAD (Audibert et al. 2020)	–	–	Reconstruction	–	–
TraceVAE (Xie et al. 2023)	–	Trace graph	Latent dynamics	–	–
MicroHECL (Liu et al. 2021)	–	Trace-induced graph	Propagation path analysis	Limited	Rule-based ranking
DiagFusion (Zhang et al. 2023)	Partial	Dependency graph	Limited	Feature-level fusion	Classification/ranking
MRCA (Wang et al. 2024)	Partial	Partial	Metric-level dependency	Multimodal representation	Metric-level RCA
CHASE (Zhao et al. 2024)	Yes	Multimodal graph	Causal hypergraph	Heterogeneous graph fusion	Hypergraph-based RCA
Topo-MFP	SBERT + transition graph	Dynamic service graph	Dilated propagation encoding	Cross-modal graph fusion	Reverse RWR

Compared with existing methods, Topo-MFP differs in three main aspects. First, it constructs a semantic-transition log template graph that jointly captures textual semantics and execution-order patterns, rather than treating log templates only as discrete event identifiers or independent text embeddings. Second, it learns a diagnosis-oriented multimodal propagation graph with a topology-aware soft sparsity prior, which uses different penalties for topology-adjacent and topology-distant service pairs. Third, it performs symptom-to-source ranking through reverse random walk with restart over the learned propagation graph, instead of applying random walk only to the raw invocation topology or relying solely on local anomaly scores.

The empirical results should be interpreted as comparisons with representative baselines that can be consistently adapted to the same service-level protocol, rather than as an exhaustive comparison with all recent RCA methods.

3 Problem formulation

A microservice system consists of a set of services:

$$V = \{v_1, v_2, \dots, v_n\}.$$

At each time window t, service v_i generates multimodal observations:

$$X_i^t = \{M_i^t, L_i^t, T_i^t\},$$

where M_i^t denotes metric features, L_i^t denotes log features, and T_i^t denotes trace-derived features.

The service invocation topology in window t is represented by an adjacency matrix:

$$A_t \in \{0,1\}^{n \times n}.$$

If service v_i invokes service v_j in window t, then:

$$A_{ij}^t = 1.$$

Otherwise:

$$A_{ij}^t = 0.$$

The invocation matrix represents the observed service communication topology. In this paper, it is used as a structural prior for topology-aware learning, while the propagation direction is further estimated from temporal and multimodal evidence.

3.1 Fault detection

Fault detection aims to determine whether a service is abnormal in a time window. Given multimodal observations and service topology, the model predicts the abnormal probability:

$$\hat{y}_i^t = f_\theta(X^t, A_t),$$

where $\hat{y}_i^t$ denotes the predicted abnormal probability of service v_i at time window t.

3.2 Root cause localization

When a fault is detected, root cause localization aims to rank service instances according to their likelihood of being root causes. Topo-MFP learns a diagnosis-oriented propagation dependency graph:

$$G_{prop} = (V, E_{prop}, A_{prop}),$$

where $A_{prop,ij}$ represents the learned influence tendency from service v_i to service v_j. A larger value means that abnormal behavior at v_i is more likely to propagate to v_j under the learned multimodal evidence and topology prior.

For RCA, symptoms are usually observed at downstream services or system-level KPIs. Therefore, root cause ranking performs reverse propagation from symptom nodes to upstream candidate nodes over the learned propagation graph.

4 Methodology

Figure 1 shows the overall architecture of Topo-MFP. The framework consists of two main components: a topology-aware multimodal fault detection component and a propagation-graph-based root cause localization component.

For fault detection, raw logs are parsed into templates. A semantic-transition log template graph is constructed using template transition probabilities and Sentence-BERT embeddings. Metrics and traces are used to construct a dynamic service dependency graph. A spatio-temporal graph attention network learns service-level representations, and cross-modal fusion combines log semantic representations with service graph representations.

For root cause localization, metrics, logs, and traces are modeled as multimodal time-series evidence. Shared-kernel dilated convolution is used to capture temporal propagation patterns, and modality-specific propagation matrices are learned from the encoded representations. Cross-modal consistency and topology-aware sparse regularization are then used to fuse propagation graphs from different modalities. Finally, reverse random walk with restart is applied to rank candidate root-cause services.

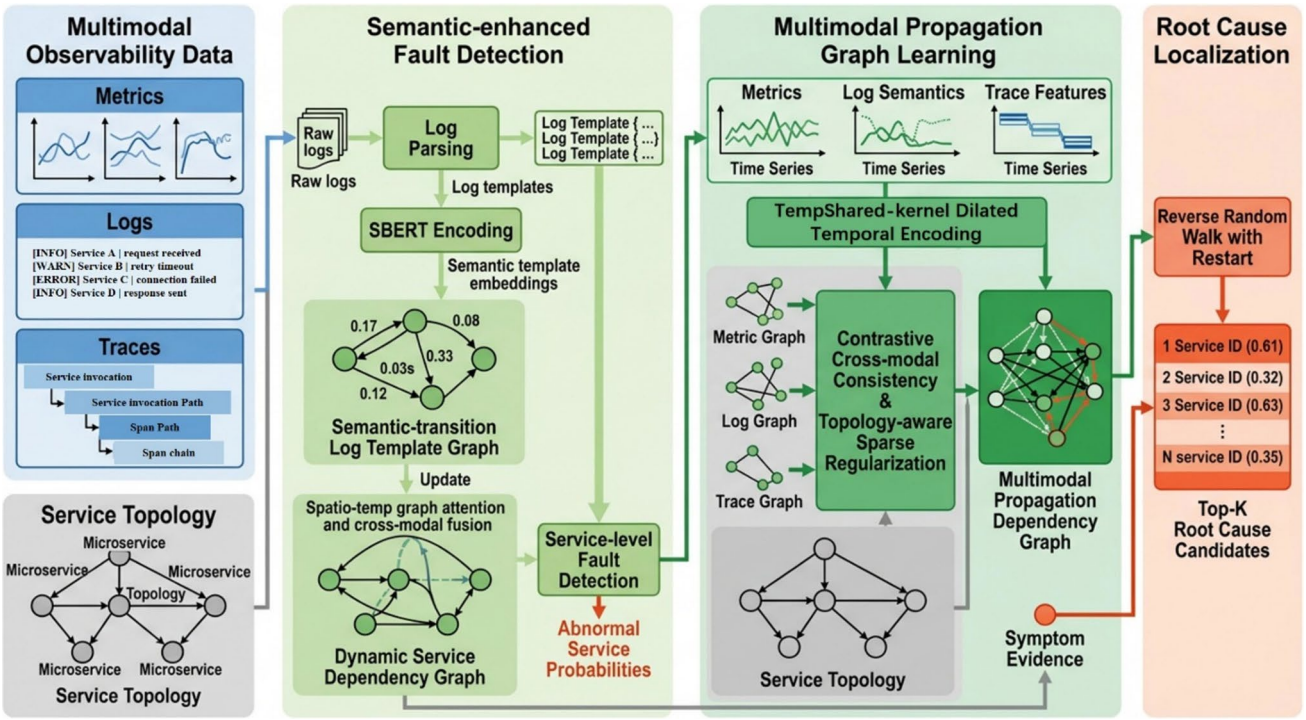


Fig. 1 Overview of the topo-MFP framework for fault detection and root cause localization

4.1 Semantic-transition log graph learning

This subsection introduces how Topo-MFP constructs and learns the semantic-transition log template graph. The goal is to jointly capture textual semantics and execution-order patterns from logs.

4.1.1 Log parsing and template semantic encoding

Raw logs contain constant text and variable parameters. We use Drain to parse raw logs into templates (He et al. 2017). Let the template set be:

$$\mathcal{T} = \{T_1, T_2, \ldots, T_m\}.$$

Each template is regarded as a node in the log template graph. To capture semantic information, each template is encoded by Sentence-BERT (Reimers and Gurevych 2019):

$$h_i^{sem} = \text{SBERT}(T_i).$$

The initial semantic feature matrix is:

$$H^{sem} = [h_1^{sem}; h_2^{sem}; \ldots; h_m^{sem}].$$

To reduce information leakage, transition statistics of the log template graph are constructed using only the training split. During validation and testing, the transition matrix is fixed. Unseen templates are encoded by SBERT but are not used to update transition statistics during evaluation (Fig. 2).

4.1.2 Log template transition graph

If template T_j appears immediately after template T_i in a training log sequence, a directed edge from T_i to T_j is added. The transition probability is defined as:

$$P(T_j|T_i) = \frac{C(T_i, T_j)}{C(T_i) + \epsilon},$$

where $C(T_i, T_j)$ is the number of times that T_j follows T_i, $C(T_i)$ is the number of occurrences of T_i, and ϵ is a smoothing constant.

The log template graph is represented as:

$$G_l = (V_l, E_l, P_l),$$

where V_l is the template node set, E_l is the transition edge set, and P_l is the transition probability matrix.

4.1.3 Gated graph attention for template representation

At layer k, the semantic attention score between template nodes i and j is:

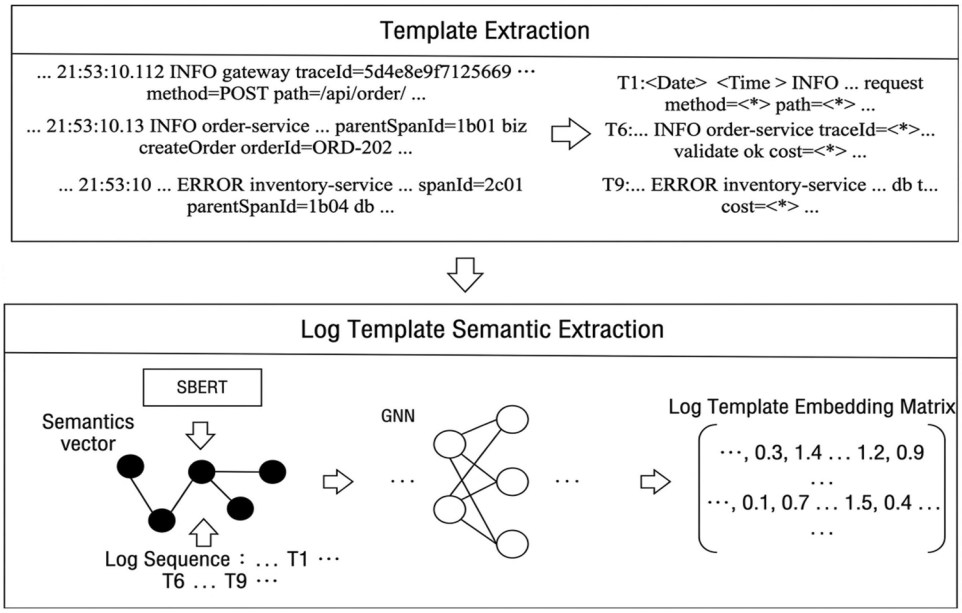


Fig. 2 Semantic-transition log template graph construction and representation learning

$$e_{ij}^k = \text{LeakyReLU}\left(a_k^T [W_k h_i^k || W_k h_j^k]\right).$$

The normalized semantic attention coefficient is:

$$\alpha_{ij}^k = \frac{\exp(e_{ij}^k)}{\sum_{r \in \mathcal{N}_i} \exp(e_{ir}^k)}.$$

A gate balances semantic attention and transition probability:

$$g_{ij}^k = \text{sigmoid}\left(w_g^T [W_g h_i^k || W_g h_j^k || P(T_j|T_i)] + b_g\right).$$

The fused attention coefficient is:

$$\beta_{ij}^k = g_{ij}^k \alpha_{ij}^k + (1 - g_{ij}^k) P(T_j|T_i).$$

The template representation is updated by:

$$h_i^{k+1} = \sigma\left(\sum_{j \in \mathcal{N}_i} \beta_{ij}^k W_k h_j^k\right).$$

After multi-layer propagation, the final template embedding is denoted as e_i^{log}.

4.2 Dynamic service dependency graph construction

This subsection describes how Topo-MFP constructs dynamic service dependency graphs from traces and aligns metrics, logs, and traces at the service-window level.

At time window t, a dynamic service dependency graph is defined as:

$$G_t = (V_t, A_t, E_t),$$

where V_t denotes service nodes, A_t denotes the adjacency matrix, and E_t denotes trace-derived edge attributes.

A directed edge from service v_i to service v_j is created if at least one trace contains an invocation from v_i to v_j in the current window. Multiple spans between the same service pair are aggregated into edge attributes, including average latency, request count, and error ratio (Fig. 3).

Metrics are standardized by Z-score normalization:

$$\bar{x} = \frac{x - \mu}{\sigma + \epsilon}.$$

For service v_i, the metric sequence in the current window is encoded by a GRU (Cho et al. 2014):

$$h_{i,m}^{\tau} = \text{GRU}(\overline{M}_i^{\tau}, h_{i,m}^{\tau-1}).$$

The metric representation is obtained by pooling hidden states:

$$z_{i,m}^t = \text{Pool}(h_{i,m}^{t-k+1}, \ldots, h_{i,m}^t).$$

For service v_i in window t, its parsed log templates form a sequence:

$$\mathcal{P}_i^t = \{p_1, p_2, \ldots, p_s\}.$$

The service-level log representation is obtained by attention pooling:

$$z_{i,l}^t = \sum_{p \in \mathcal{P}_i^t} \gamma_{ip}^t e_p^{log}.$$

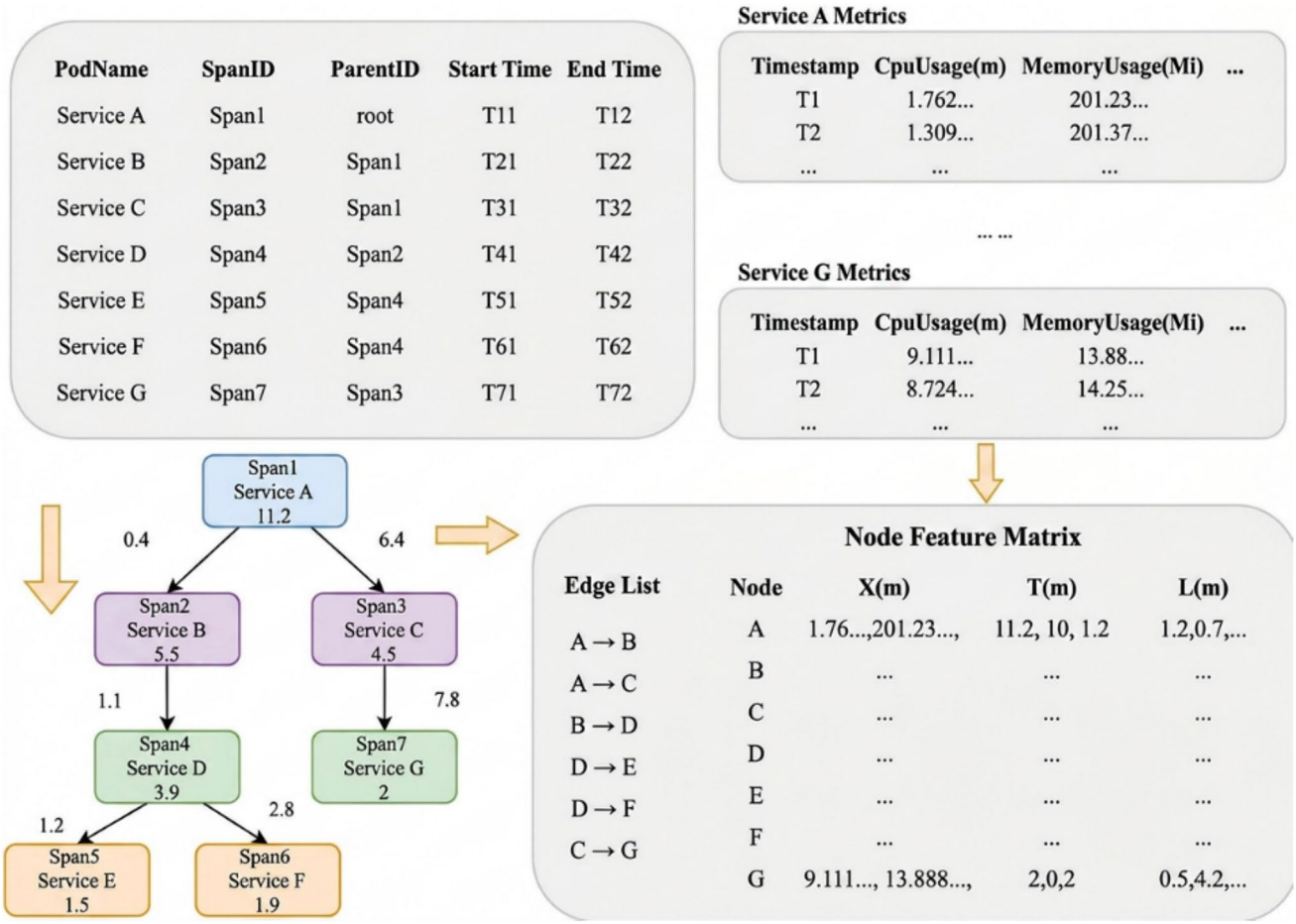


Fig. 3 Construction of the dynamic service dependency graph from traces and alignment of service-level metric, log, and trace features

The attention weight γ_{ip}^{t} is computed by a service-window-specific query so that templates more relevant to the current service state receive larger weights. Specifically,

$$a_{ip}^{t} = u_l^T \tanh\left(W_l e_p^{log} + W_m z_{i,m}^{t} + W_{tr} z_{i,tr}^{t} + b_l\right).$$

The normalized template attention weight is

$$\gamma_{ip}^{t} = \frac{\exp(a_{ip}^{t})}{\sum_{p' \in \mathcal{P}_i^t} \exp(a_{ip'}^{t})}.$$

This design makes service-level log aggregation dependent not only on template semantics but also on the metric and trace context of the same service window.

Trace-derived features include average response time, request count, and error ratio. These features are normalized and encoded as:

$$z_{i,tr}^{t} = \sigma(W_{tr} q_i^{t} + b_{tr}),$$

where q_i^{t} denotes trace-derived statistical features.

The initial node representation of service v_i is:

$$h_i^{svc,t} = [z_{i,m}^{t} || z_{i,l}^{t} || z_{i,tr}^{t}].$$

It is projected into a unified hidden space:

$$x_i^{t} = \sigma(W_x h_i^{svc,t} + b_x).$$

4.3 Topology-aware fault detection

This subsection presents the topology-aware fault detection module. It learns service-level abnormal representations by combining multimodal service features and dynamic service dependency graphs.

For service v_i and neighbor v_j, the spatial attention score is:

$$u_{ij}^{svc} = a_{svc}^{T} \text{LeakyReLU}\left(W_{svc}[x_i^{t} || x_j^{t} || e_{ij}^{t}]\right),$$

where e_{ij}^{t} is the trace-derived edge feature.

The normalized attention coefficient is:

$$\alpha_{ij}^{svc} = \frac{\exp(u_{ij}^{svc})}{\sum_{k\in\mathcal{N}_i}\exp(u_{ik}^{svc})}.$$

The graph-aware node representation is:

$$z_{i,g}^{t} = \sigma\left(\sum_{j\in\mathcal{N}_i}\alpha_{ij}^{svc}W_{svc}x_j^t\right).$$

Graph-aware node representations over a sliding window are aggregated by temporal attention (Vaswani et al. 2017):

$$Z_i^{t-k+1:t} = [z_{i,g}^{t-k+1},\ldots,z_{i,g}^{t}].$$

The temporal representation is:

$$z_{i,T}^{t} = \text{MultiHeadAttention}(Z_i^{t-k+1:t}).$$

For cross-modal fusion, the key and value are the sequence of template embeddings generated by the service in the current window. Let:

$$Z_{i,l}^{t} = [e_{p_1}^{log}, e_{p_2}^{log},\ldots,e_{p_s}^{log}].$$

The log context guided by the service representation is:

$$\widehat{z}_{i,l}^{t} = \text{softmax}\left(\frac{(W_Q z_{i,T}^{t})(W_K Z_{i,l}^{t})^T}{\sqrt{d}}\right)W_V Z_{i,l}^{t}.$$

The fused representation is:

$$z_i^t = \text{LayerNorm}\left(z_{i,T}^{t} + \widehat{z}_{i,l}^{t}\right).$$

The predicted abnormal probability is:

$$\widehat{y}_i^t = \text{sigmoid}(W_o z_i^t + b_o).$$

The binary cross-entropy loss is:

$$L_{det} = -\frac{1}{|\mathcal{B}|}\sum_{(i,t)\in\mathcal{B}}[y_i^t\log\widehat{y}_i^t + (1-y_i^t)\log(1-\widehat{y}_i^t)].$$

A contrastive alignment loss is introduced to align service graph representations and log semantic representations (Oord et al. 2018):

$$L_{align} = -\frac{1}{|\mathcal{B}|}\sum_{(i,t)\in\mathcal{B}}\log\frac{\exp(\text{sim}(z_{i,g}^{t}, z_{i,l}^{t})/\tau)}{\sum_{(j,t')\in\mathcal{B}}\exp(\text{sim}(z_{i,g}^{t}, z_{j,l}^{t'})/\tau)}.$$

The final fault detection loss is:

$$L_{FD} = L_{det} + \lambda_{align}L_{align}.$$

4.4 Diagnosis-oriented propagation graph learning

This subsection introduces the RCA module. Unlike the fault detection module, which predicts service-level abnormal probabilities, the RCA module learns a diagnosis-oriented propagation dependency graph for symptom-to-source ranking.

4.4.1 Training and inference protocol for RCA

Topo-MFP separates propagation graph learning from inference-time root cause ranking. During training, the RCA module learns temporal propagation representations from historical multimodal observations using next-window prediction, cross-modal consistency, and topology-aware sparsity. During inference, the model only uses observations available up to the diagnosis window. Future-window observations and ground-truth root cause labels are not used for propagation graph construction, restart vector initialization, or final ranking.

For a diagnosis window

t

the input to the RCA module is the historical observation window

$$\mathcal{W}_t = \{t-w+1, t-w+2,\ldots,t\}.$$

The output is a ranked list of candidate services

$$\mathcal{C}_t \subseteq V,$$

where $\mathcal{C}_t$ contains services involved in the diagnosis window. The service-level candidate set is constructed from services that appear in traces, emit logs, or report metric observations within $\mathcal{W}_t$. No service is removed only because its local anomaly score is low.

4.4.2 Temporal propagation representation learning

The learned graph in Topo-MFP is a diagnosis-oriented propagation dependency graph. It is learned from temporal precedence, multimodal abnormal correlations, and service topology priors, and is used to support service-level root cause ranking (Fig. 4).

For modality $p \in \{m, l, tr\}$, the temporal representation is:

$$H_p^t = \sigma\left(W_p *_d X_p^t + \gamma_p W_s *_d X_p^t\right),$$

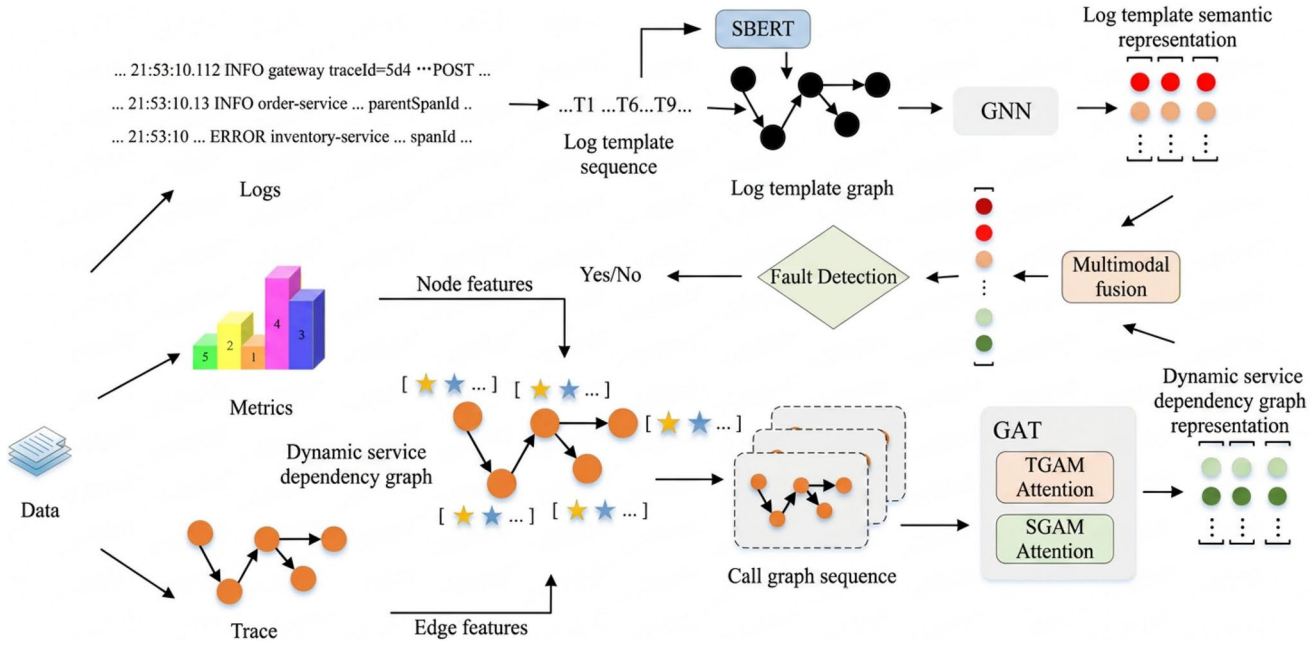


Fig. 4 Propagation-graph-based root cause localization framework of Topo-MFP

where $*_d$ denotes dilated convolution, W_p is a modality-specific kernel, W_s is the shared kernel, and γ_p is a learnable scaling factor.

The next-window prediction objective is used only during training. During inference, Topo-MFP uses observations available up to the diagnosis window and does not use future-window observations for RCA initialization or ranking.

For modality p, the propagation score between service v_i and v_j is generated by:

$$s_{ij,p}^{t} = \mathrm{MLP}_p\left(\left[H_{i,p}^{t} \parallel H_{j,p}^{t} \parallel A_{ij}^{topo}\right]\right).$$

The propagation dependency weight is:

$$A_{ij,p}^{t} = \sigma\left(s_{ij,p}^{t}\right).$$

Here, $A_{ij,p}^{t}$ represents the propagation tendency from service v_i to service v_j under modality p.

In propagation score computation, Topo-MFP considers service pairs within the diagnosis-window candidate set so that topology-distant dependencies can still be assigned non-zero weights when supported by multimodal evidence. The topology-aware sparsity prior encourages topology-distant propagation weights to become small, but no explicit top-k or threshold-based pruning is applied in the current implementation. In message passing, $N_{\mathrm{i}}^{\mathrm{prop}}$ denotes the candidate services in the diagnosis window, excluding v_{i} itself.

Graph message passing is performed as:

$$m_{i,p}^{t} = \sum_{j \in N_{\mathrm{i}}^{\mathrm{prop}}} A_{ji,p}^{t} W_c H_{j,p}^{t}$$

The node representation is updated by:

$$\widetilde{H}_{i,p}^{t} = \sigma\left(W_h\left[H_{i,p}^{t} \parallel m_{i,p}^{t}\right]\right).$$

The prediction loss is:

$$\mathcal{L}_{pred} = \frac{1}{n}\sum_{i=1}^{n} \parallel \hat{x}_i^{t+1} - x_i^{t+1} \parallel_2^2.$$

4.4.3 Contrastive cross-modal consistency

For two modalities p and q, a positive pair is defined as representations of the same service in the same time window and fault case. Negative pairs are sampled from different services, different windows, or different fault cases within the mini-batch.

The InfoNCE-style objective is:

$$\mathcal{L}_{CC}^{p,q} = -\frac{1}{|\mathcal{B}|}\sum_{(i,t)\in\mathcal{B}} \log \frac{\exp\left(\mathrm{sim}\left(H_{i,p}^{t}, H_{i,q}^{t}\right)/\tau\right)}{\sum_{(j,t')\in\mathcal{B}} \exp\left(\mathrm{sim}\left(H_{i,p}^{t}, H_{j,q}^{t'}\right)/\tau\right)}.$$

The overall contrastive consistency loss is:

$$\mathcal{L}_{CC} = \sum_{p \neq q} \mathcal{L}_{CC}^{p,q}.$$

This objective does not claim to estimate exact mutual information. Instead, it provides a contrastive consistency signal to encourage different modalities to support compatible propagation patterns.

4.4.4 Propagation graph fusion and topology-aware regularization

For an abnormal diagnosis window t, Topo-MFP fuses modality-specific propagation matrices within the corresponding historical window $\mathcal{W}_t$:

$$A_p^{\mathcal{W}_t} = \frac{1}{|\mathcal{W}_t|} \sum_{\tau \in \mathcal{W}_t} A_p^{\tau}.$$

A graph-level modality score is computed as:

$$\omega_p^t = \frac{\exp(w_f^T \mathrm{Pool}(H_p^t))}{\sum_{q \in \{m,l,tr\}} \exp(w_f^T \mathrm{Pool}(H_q^t))}.$$

Thus:

$$\omega_m^t + \omega_l^t + \omega_{tr}^t = 1.$$

The fused propagation graph for the diagnosis window is:

$$A_{prop}^t = \omega_m^t A_m^{\mathcal{W}_t} + \omega_l^t A_l^{\mathcal{W}_t} + \omega_{tr}^t A_{tr}^{\mathcal{W}_t}.$$

Let A^{topo} be the service invocation adjacency matrix. Topology is used as a soft structural prior. A weighted sparse regularization term is introduced:

$$L_{topo} = \sum_{i,j} \rho_{ij} |A_{prop,ij}^t|,$$

where:

$$\rho_{ij} = \begin{cases} \rho_{in}, & A_{ij}^{topo} = 1 \text{ or } A_{ji}^{topo} = 1, \\ \rho_{out}, & \text{otherwise.} \end{cases}$$

Here, $\rho_{out} > \rho_{in}$. This design encourages the learned propagation graph to prefer topology-adjacent service pairs while still allowing implicit dependencies when data evidence is strong. The bidirectional adjacency condition is used because invocation direction and fault propagation direction are not always identical in microservice systems.

The final RCA learning objective is:

$$L_{RCA} = L_{pred} + \lambda_1 L_{CC} + \lambda_2 L_{topo} + \lambda_3 \parallel A_{prop}^t \parallel_1.$$

4.5 Reverse random walk with restart for RCA

This subsection presents the inference-time root cause ranking procedure. The edge direction in A_{prop}^t follows the learned propagation direction. Specifically, $A_{prop,ij}^t$ means that abnormal behavior at service v_i may influence service v_j.

Since observed symptoms are often located at downstream services or system KPIs, RCA should trace backward from symptoms to upstream candidates. Therefore, Topo-MFP constructs a reverse transition matrix from $\left(A_{prop}^t\right)^T$. Reverse random walk with restart is based on graph-based proximity ranking (Tong et al. 2006).

Let P^{rev} be the normalized reverse transition matrix derived from $\left(A_{prop}^t\right)^T$. The iteration is:

$$r^{k+1} = (1-\alpha) P^{rev} r^k + \alpha r_0,$$

where $r_0$is initialized by abnormal service scores or KPI-related symptom nodes, and $\alpha$is the restart probability. After convergence, services are ranked according to their stationary probabilities.

The learned propagation graph should be interpreted as a diagnosis-oriented ranking representation rather than as a verified causal graph. The reverse RWR procedure uses this representation to rank root-cause candidates by tracing symptoms back to likely upstream sources.

4.6 Algorithm summary

To make the workflow of Topo-MFP clearer, Algorithms 1 and 2 summarize the fault detection and root cause localization procedures, respectively.

Algorithm 1: Fault Detection

Algorithm 1. Topology-aware Multimodal Fault Detection	
Require	Logs, metrics, traces, service topology, window size
Ensure	Service-level abnormal probabilities
1	Parse raw logs into templates using Drain.
2	Encode log templates with Sentence-BERT.
3	Build the semantic-transition log template graph from training log sequences.
4	Learn template embeddings using gated graph attention.
5	Normalize service metrics and encode metric sequences using GRU.
6	Aggregate service-level log embeddings through attention pooling.
7	Extract trace-derived node features and edge attributes.
8	Construct dynamic service dependency graphs for diagnosis windows.
9	Compute topology-aware service representations using spatial graph attention.
10	Aggregate temporal service representations using temporal attention.
11	Fuse service graph representations with log semantic representations.
12	Compute abnormal probabilities using a sigmoid prediction layer.
13	**return** service-level abnormal probabilities.

Algorithm 2: Root Cause Localization

Algorithm 2. Propagation-graph-based Root Cause Localization	
Require	Multimodal time series, service topology, abnormal diagnosis window
Ensure	Ranked root-cause candidate services
1	Construct the historical observation window before the diagnosis time.
2	Encode metric, log, and trace sequences using shared-kernel dilated convolution.
3	Learn modality-specific propagation matrices from encoded representations.
4	Compute cross-modal contrastive consistency among modality-specific representations.
5	Fuse modality-specific propagation matrices with learned modality weights.
6	Apply topology-aware sparse regularization with different penalties for topology-adjacent and topology-distant service pairs.
7	Obtain the fused diagnosis-oriented propagation graph.
8	Build the reverse transition matrix from the transposed propagation graph.
9	Initialize the restart vector using abnormal service scores or KPI-related symptom nodes.
10	**repeat**
11	Update ranking scores by reverse random walk with restart.
12	**until** convergence or maximum iterations are reached.
13	Rank candidate services by the stationary probabilities.
14	**return** ranked root-cause candidate services.

5 Experimental setup

5.1 Experimental environment

The experiments are conducted on a Windows 11 64-bit operating system. The programming language is Python 3.9, and the models are implemented using PyTorch 1.13 and PyTorch Geometric (Paszke et al. 2019). The hardware platform contains an AMD Ryzen series CPU, an NVIDIA GeForce RTX 4090 GPU with 24 GB memory, and 64 GB system memory. CUDA 11.7 is used for GPU acceleration (Table 2).

5.2 Datasets

Two multimodal microservice datasets are used in the experiments: AIOps2022 and a TrainTicket-based TT dataset. Both datasets contain metrics, logs, and distributed traces, and their original fault labels are provided at the operation level.

Table 2 Experimental environment

Item	Configuration
Programming language	Python 3.9
Deep learning framework	PyTorch 1.13
Operating system	Windows 11 64-bit
CUDA	11.7
CPU	AMD Ryzen series
GPU	NVIDIA GeForce RTX 4090
GPU memory	24 GB
Memory	64 GB

AIOps2022 is derived from the CCF International AIOps Challenge 2022 and is based on an industrial microservice e-commerce scenario. The system covers typical business procedures such as user access, product query, order processing, and payment. Faults are injected through chaos engineering and include common production failures such as CPU overload, memory leakage, network delay, packet loss, and database connection failure. The dataset provides service metrics, logs, and traces collected from an industrial-style microservice environment.

The TT dataset is constructed based on the open-source TrainTicket benchmark, a representative microservice system for train-ticket reservation scenarios. The system is deployed in a Docker-based environment and contains 41 microservices, among which 27 services directly support core business functions. Runtime observability data are collected using commonly used monitoring tools, including Prometheus and cAdvisor for metrics, Jaeger for distributed traces, and service-level logging components for logs. Before fault injection, the system is executed under normal workload for approximately 7 h to collect fault-free baseline data. User requests are generated to cover representative business operations, including ticket search, user login, order creation, payment, route query, and ticket cancellation. Faults are injected using Chaosblade to reproduce typical microservice failures including CPU resource exhaustion, network congestion, random packet loss, and service-level delay. Each fault injection case contains a warm-up period, a fault injection period, and a recovery period. Metrics, logs, and traces are collected continuously during the whole process and aligned into fixed-length diagnosis windows.

Table 3 reports partial statistics of the two datasets. These statistics are computed from the processed datasets used in our experiments and are reported to characterize the trace structure and label granularity.

Table 4 summarizes the TT fault injection settings. The numbers of injections and workload parameters are reported to make the dataset construction more transparent.

5.3 Label mapping and candidate set construction

The original labels in both datasets are operation-level labels. Since the compared methods have different output granularities and several baselines cannot directly produce operation-level rankings, this paper adopts a unified service-level evaluation protocol. Specifically, each faulty operation is mapped to the service that owns or executes the operation. If multiple faulty operations belong to the same service in a diagnosis case, they are merged into one service-level root-cause label.

This mapping makes the evaluation protocol consistent across compared methods, but it also reduces the granularity of localization. In particular, when multiple operations are mapped to the same service, the service-level RCA task may be easier than operation-level localization. Therefore, the RCA results in this paper should be interpreted as service-level localization performance rather than operation-level diagnosis performance. Table 5 reports operation-to-service mapping statistics. Table 6 further reports case-level mapping statistics for RCA evaluation.

For each RCA case, the candidate set consists of services that appear in the corresponding diagnosis window through traces, logs, or metric observations. No service is removed solely because its local anomaly score is low. This design avoids artificially simplifying the ranking task by filtering out low-score services before ranking. However, the reported RCA metrics are computed over diagnosis-window candidate sets rather than over all deployed services. Therefore, the results should not be interpreted as whole-system ranking performance. Because the diagnosis-window candidate sets are moderate in size, top-K metrics, especially Hit@3 and Hit@5, should be interpreted together with candidate-set size distributions and random-ranking results.

Table 7 reports the candidate-set size distribution after service-level label mapping. We report both the overall distribution and the test-split distribution. The overall distribution characterizes the scale of the service-level RCA task, while the test-split distribution directly corresponds to the RCA results reported in Tables 18 and 19. The candidate set of each RCA case consists of services that appear in traces, emit logs, or report metric observations within the corresponding diagnosis window.

Table 3 Partial statistics of the datasets

Dataset	Avg. depth	Avg. spans	Fault granularity	Avg. services	Avg. operations
AIOps2022	3.2	39.0	Operation-level	5.9	7.6
TT	4.2	21.7	Operation-level	6.9	11.6

Table 4 Fault injection settings of the TT dataset

Fault type	Injected target	# Injections	Duration per injection	Workload setting
CPU resource exhaustion	Service/container level	30	10 min	80 concurrent users/about 120 requests per second
Network congestion/delay	Service/container level	24	10 min	80 concurrent users/about 120 requests per second
Random packet loss	Service/container level	24	10 min	80 concurrent users/about 120 requests per second
Service-level delay/timeout	Service/API level	18	10 min	80 concurrent users/about 120 requests per second
Database or middleware disturbance	Service/container level	12	10 min	80 concurrent users/about 120 requests per second
Total	–	108	–	–

Table 5 Operation-to-service label mapping statistics

Dataset	# Services	# Operations	Avg. operations per service	Max operations per service	Services with more than one operation
AIOps2022	32	61	1.91	5	18
TT	41	96	2.34	7	27

Table 6 Case-level operation-to-service mapping statistics

Dataset	# RCA cases	Cases with one-to-one mapping	Cases with many-to-one mapping	Many-to-one ratio
AIOps2022	156	109	47	30.1%
TT	108	72	36	33.3%

The expected performance of uniformly random ranking is computed to provide a descriptive lower-bound reference under the candidate-set size distributions. When each case contains one service-level ground-truth root cause, the expected random Hit@K is computed as:

$$\mathrm{E[Hit@K]} = \frac{1}{N}\sum_{i=1}^{N} \min\left(\frac{K}{C_i}\right), 1$$

where C_i denotes the number of candidate services in the i-th RCA case. Based on the candidate-set size distributions reported in Table 7, Table 8 reports the expected random-ranking performance for both all RCA cases and the test split. The test-split Random reference is used when interpreting the final RCA results in Tables 18 and 19. The expected MAP@3 is computed under the same uniform random-ranking assumption. This reference is used to illustrate the influence of moderate candidate-set sizes on top-K metrics.

5.4 Data splitting and leakage control

For both datasets, the data are divided into training, validation, and testing splits at the fault-case or fault-interval level, rather than by randomly splitting individual sliding windows. This protocol is used to reduce temporal leakage caused by overlapping windows.

For TT, the grouping unit is a fault injection case. All windows derived from the same injection case are assigned to the same split. The normal baseline period is also divided chronologically into non-overlapping blocks, and windows from the same block are assigned to only one split. The dataset contains 108 fault injection cases, among which 70 cases are used for training, 16 for validation, and 22 for testing. This corresponds approximately to a 65%/15%/20% split at the case level.

For AIOps2022, when annotated fault-interval metadata are available, the grouping unit is the annotated fault interval. For records without complete interval metadata, windows are grouped into chronological fault blocks according to their timestamps and labels. The boundary of a chronological fault block is determined by changes in fault labels and temporal gaps between adjacent windows. Windows overlapping the same fault interval are assigned to only one split. When a window overlaps two intervals or cannot be assigned to a complete interval, it is excluded from repeated-run evaluation to avoid ambiguous leakage. The remaining windows are grouped chronologically by fault intervals and split into training, validation, and testing groups. The dataset contains 156 RCA fault cases, among which 101 cases are used for training, 23 for validation, and 32 for testing. Table 9 summarizes the split protocol.

In addition to the fault-case-level split, Table 10 reports the window-level statistics used for fault detection and RCA evaluation. Normal windows are used for fault detection training and evaluation, whereas RCA evaluation is conducted only on fault cases with service-level root-cause labels. Diagnosis windows are constructed from the processed time-window sequence with a historical window length of 20 time steps and a sliding stride of 1 time step. Windows crossing split boundaries or overlapping ambiguous fault intervals are excluded before repeated-run evaluation.

The same split protocol is used for all compared methods. Hyperparameters are selected on the validation split, and the test split is used only for final reporting. The following preprocessing components are fitted using only the training split: feature normalization statistics, log template transition statistics, log template graph transition probabilities, and model-selection hyperparameters. During validation and testing, these components are frozen. Unseen log templates are encoded by SBERT, but they are not used to update the log transition matrix during evaluation.

The next-window prediction objective used in the RCA module is applied only during training. During inference, Topo-MFP uses observations available up to the diagnosis window and does not use future-window observations, test-set labels, or ground-truth root causes for propagation graph construction, restart vector initialization, or final ranking.

5.5 Baselines

For fault detection, three representative baselines are selected: UAC-AD (Liu et al. 2024), TraceVAE (Xie et al. 2023), and USAD (Audibert et al. 2020). UAC-AD is a multimodal anomaly detection method based on adversarial contrastive learning. TraceVAE is a trace-based graph variational autoencoder for microservice trace anomaly detection. USAD is an autoencoder-based unsupervised anomaly detection method for multivariate time series.

For root cause localization, MRCA (Wang et al. 2024), MicroHECL (Liu et al. 2021), and DiagFusion (Zhang et al. 2023) are selected as representative learning-based or multimodal RCA baselines. MRCA performs metric-level root cause analysis with multimodal data. MicroHECL analyzes abnormal propagation paths on dynamic service dependency graphs. DiagFusion performs multimodal failure diagnosis using dependency information. These RCA baselines are selected because they can be consistently adapted to the same service-level label mapping, diagnosis-window candidate-set construction, and available multimodal inputs. Several recent RCA methods, such as CHASE, RUN, DGERCL, Grace, and MicroEGRCL, are discussed in the related work but are not included in the main comparison because their required inputs, graph construction assumptions, intervention settings, or output granularity differ from the evaluation protocol adopted in this paper.

To address the effect of candidate-set size and the reverse random walk mechanism, additional reference and controlled baselines are considered.

Random is computed as the expected performance of uniformly random ranking over the candidate services, rather

Table 7 Candidate-set size distribution for RCA cases

Dataset	Split	# RCA cases	Min	Q1	Median	Mean	Q3	Max
AIOps2022	All	156	3	5	6	6.2	7	11
AIOps2022	Test	32	3	5	6	6.1	7	10
TT	All	108	4	6	7	7.1	8	13
TT	Test	22	4	6	7	7.0	8	12

Table 8 Expected random-ranking performance based on candidate-set sizes

Dataset	Split	Expected Hit@1	Expected Hit@3	Expected Hit@5	Expected MAP@3
AIOps2022	All	0.166	0.491	0.772	0.290
AIOps2022	Test	0.169	0.497	0.779	0.294
TT	All	0.145	0.428	0.694	0.253
TT	Test	0.147	0.434	0.702	0.256

Table 9 Dataset split protocol and leakage-control grouping

Dataset	Grouping unit	# Groups/cases	Training	Validation	Testing	Cross-split windows
AIOps2022	Fault interval/ chronological fault block	156	101	23	32	0
TT	Fault injection case/normal chronological block	108	70	16	22	0

Table 10 Window-level split statistics

Dataset	Split	# Windows	# Fault windows	# Normal windows	# RCA cases	# Excluded ambiguous windows
AIOps2022	Training	7420	3186	4234	101	38
AIOps2022	Validation	1685	724	961	23	9
AIOps2022	Testing	2360	1008	1352	32	14
TT	Training	5040	3360	1680	70	0
TT	Validation	1152	768	384	16	0
TT	Testing	1584	1056	528	22	0

than by training a model. It provides a descriptive lower-bound reference for interpreting Hit@K under moderate candidate-set sizes.

Local-Score is a controlled baseline using the same trained fault detection module as Topo-MFP. It ranks candidate services only according to their local abnormal scores and does not use propagation graph learning or reverse RWR. The abnormal scores are predicted without using test-set labels. This baseline evaluates whether RCA can be solved by local anomaly intensity alone. Topology-RWR performs reverse random walk with restart over the raw invocation topology.

For each diagnosis window, the raw invocation topology is constructed from invocation edges observed within the same historical observation window W_t. It does not use future traces, validation/test labels, or ground-truth root causes. Topology-RWR uses the same candidate set, restart probability, convergence criterion, and anomaly-score-based restart vector as Topo-MFP. The only difference is that Topology-RWR uses the raw invocation adjacency matrix instead of the learned multimodal propagation graph. Therefore, the comparison between Topology-RWR and Topo-MFP isolates the contribution of the learned propagation graph from the reverse RWR mechanism itself.

Let A_{topo} be the raw invocation adjacency matrix, where $A_{\text{topo},ij} = 1$ indicates that service v_i invokes service v_j. Topology-RWR constructs a reverse transition matrix from the transpose of A_{topo} and updates the ranking score as:

$$r^{(k+1)} = (1-\alpha)P_{\text{topo}}^{\text{rev}} r^{(k)} + \alpha r_0$$

where $P_{\text{topo}}^{\text{rev}}$ is the normalized reverse transition matrix, r_0 is initialized by service-level abnormal scores, and α is the restart probability. Table 11 summarizes the baseline methods used in the experiments.

5.6 Evaluation metrics

For fault detection, precision, recall, F1-score, and AUC are used. Precision measures the proportion of predicted abnormal samples that are truly abnormal, while recall measures the proportion of true abnormal samples that are correctly detected. F1-score is the harmonic mean of precision and recall. AUC evaluates the ranking quality of abnormal scores under different thresholds.

For root cause localization, Hit@K and MAP@K are used. Hit@K measures whether at least one ground-truth root-cause service appears in the top-K ranked candidate list. This metric is sometimes referred to as top-K localization accuracy in RCA studies. To avoid ambiguity with precision at K, this paper consistently uses the term Hit@K.

For the i-th RCA case, let G_i be the set of ground-truth service-level root causes and R_i^K be the top-K ranked candidate services. Hit@K is defined as:

$$Hit@K = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\left(G_i \cap R_i^K \neq \varnothing\right)$$

MAP@K evaluates the ranking quality by considering the position of ground-truth root causes in the ranked list:

$$\text{AP}_i@K = \frac{1}{\min(K, |\ G_i\ |)} \sum_{j=1}^{K} P_i@j \cdot \text{rel}_i(j)$$

$$\text{MAP@K} = \frac{1}{N}\sum_{i=1}^{N} \text{AP}_i@K$$

where $\text{rel}_i(j)$ indicates whether the j-th ranked service is a ground-truth root cause, and $P_i@j$ is the precision at rank j.

Because the candidate sets in the evaluated datasets are moderate in size, Hit@3 and Hit@5 may be easier to saturate. Therefore, this paper reports candidate-set size distributions and random-ranking results, and places more emphasis on Hit@1 and MAP@3 when interpreting RCA performance.

5.7 Hyperparameter settings

Table 12 reports the main hyperparameters used in Topo-MFP. These values are selected based on the validation split and kept fixed across the main experiments unless otherwise specified.

The optimizer is fixed as AdaBelief in all experiments. The hidden dimensions are specified by module rather than

described as a range. The default historical window size is 20, based on the parameter sensitivity results. The restart probability α is set to 0.15 for both Topology-RWR and Topo-MFP to ensure a controlled comparison.

6 Experimental results

6.1 Repeated-run evaluation protocol

To evaluate the robustness of the main comparison results, we report the mean and standard deviation over five random seeds: 2022, 2023, 2024, 2025, and 2026. The repeated-run evaluation is conducted for the main fault detection and root cause localization comparisons. For fault detection, precision, recall, F1-score, and AUC are computed for each seed. For root cause localization, Hit@1, Hit@3, Hit@5, and MAP@3 are computed for each seed.

We additionally conduct two-sided paired Wilcoxon signed-rank tests over the five seeds between Topo-MFP and the strongest corresponding baseline. For fault detection, Topo-MFP is compared with TraceVAE. For root cause localization, Topo-MFP is compared with Topology-RWR. Because only five seeds are used, the tests have limited statistical power and are used as supplementary evidence. The main interpretation therefore focuses on mean ± standard deviation and the consistency of improvements across repeated runs (Table 13).

Ablation studies, log encoder comparisons, and parameter sensitivity analyses are conducted under the default experimental setting to analyze module contributions and parameter effects.

6.2 Fault detection results

Tables 14 and 15 report the fault detection results on AIOps2022 and TT, respectively. The results are presented as mean ± standard deviation over five random seeds.

Topo-MFP obtains the highest mean F1-scores and AUC values among the evaluated methods on both datasets. On AIOps2022, Topo-MFP achieves an F1-score of 0.949 ± 0.004, outperforming the strongest baseline TraceVAE by 0.015 in mean F1-score, and improves AUC from 0.932 ± 0.005 to 0.956 ± 0.003. On TT, Topo-MFP achieves an F1-score of 0.951 ± 0.004, improving over TraceVAE by 0.011 in mean F1-score, and increases AUC from 0.933 ± 0.004 to 0.963 ± 0.004. These results indicate that semantic log modeling and topology-aware multimodal representation learning provide useful complementary information for service-level fault detection.

The improvement over TraceVAE is moderate but stable across repeated runs. Therefore, the detection results should be interpreted as evidence of a consistent improvement trend rather than as a large-margin performance gap.

6.3 Fault detection ablation study

Tables 16 and 17 report the fault detection ablation results under the default experimental setting. These ablation studies are used to analyze the contribution of different input modalities and model modules, while the repeated-run statistics for the complete model and main baselines are reported in Tables 14 and 15.

Removing log semantics causes the largest degradation, with F1-score decreasing from 0.949 to 0.889. This indicates that log semantic information plays an important role in detecting abnormal execution behaviors. Removing traces also reduces precision and F1-score, suggesting that trace-derived service dependencies help constrain abnormal evidence over the service topology. Removing metrics leads to a smaller but still observable degradation, showing that resource-level signals provide complementary runtime evidence.

SGAM denotes the spatial graph attention module, TGAM denotes the temporal graph attention module, and

Table 11 Baseline methods used in the experiments

Method	Task	Input modality	Main idea
USAD	Fault detection	Metrics	Autoencoder-based time-series anomaly detection
UAC-AD	Fault detection	Multimodal	Adversarial contrastive multimodal anomaly detection
TraceVAE	Fault detection	Traces	Graph VAE for trace anomaly detection
Random reference	RCA	None	Expected performance of uniform random ranking over candidate services
Local-Score	RCA	Metrics, logs, traces	Ranking by local abnormal scores only
Topology-RWR	RCA	Raw invocation topology	Reverse RWR over raw topology with anomaly-score restart
MicroHECL	RCA	Metrics, traces	Propagation path analysis on service dependency graph
DiagFusion	RCA	Multimodal	Multimodal failure diagnosis with dependency information
MRCA	RCA	Multimodal	Metric-level root cause analysis with multimodal data
Topo-MFP	RCA/fault detection	Metrics, logs, traces	Learned multimodal propagation graph and symptom-to-source reverse RWR

Table 12 Main hyperparameter settings

Category	Hyperparameter	Value
Optimization	Optimizer	AdaBelief
Optimization	Learning rate	0.001
Optimization	Batch size	50
Optimization	Dropout	0.2
Optimization	Maximum epochs	100
Optimization	Early stopping patience	5 epochs
Windowing	Historical window size	20
Representation	Log semantic dimension for detection	512
Representation	Log semantic dimension for RCA	256
Representation	Metric GRU hidden dimension	128
Representation	Service graph hidden dimension	256
Representation	Propagation graph hidden dimension	256
Contrastive learning	Temperature τ	0.2
Fault detection loss	λ_align	0.06
RCA loss	λ_1 for cross-modal consistency	0.10
RCA loss	λ_2 for topology-aware sparsity	0.01
RCA loss	λ_3 for L1 sparsity	0.001
Topology prior	ρ_in	0.10
Topology prior	ρ_out	1.00
RWR	Restart probability α	0.15
RWR	Maximum iterations	100
RWR	Convergence tolerance	1e-6

Table 13 Supplementary significance tests for the main repeated-run comparisons

Task	Dataset	Metric	Compared baseline	*p*-value
Fault detection	AIOps2022	F1-score	TraceVAE	0.063
Fault detection	AIOps2022	AUC	TraceVAE	0.063
Fault detection	TT	F1-score	TraceVAE	0.125
Fault detection	TT	AUC	TraceVAE	0.063
RCA	AIOps2022	Hit@1	Topology-RWR	0.063
RCA	AIOps2022	MAP@3	Topology-RWR	0.063
RCA	TT	Hit@1	Topology-RWR	0.063
RCA	TT	MAP@3	Topology-RWR	0.063

The *p*-values are computed using two-sided paired Wilcoxon signed-rank tests over five random seeds. Because only five seeds are used, these tests are reported as supplementary evidence. In the result analysis, we avoid relying solely on p-values and instead emphasize repeated-run stability and effect size

CGAM denotes the cross-modal guided attention module. Removing TGAM causes the largest performance drop among the three module-level variants, indicating that temporal evolution is important for fault detection. Removing SGAM weakens service topology modeling, while removing CGAM reduces cross-modal alignment. These results suggest that SGAM, TGAM, and CGAM provide complementary contributions.

6.4 Root cause localization results

Tables 18 and 19 report the root cause localization results on AIOps2022 and TT, respectively. To address the effect of candidate-set size, local anomaly scores, and the reverse random walk mechanism, we include a Random reference and two controlled baselines: Local-Score and Topology-RWR. The Random reference reports the expected performance of uniform random ranking computed from the test-split candidate-set sizes. Local-Score ranks services only according to their local abnormal scores. Topology-RWR applies the same reverse random walk with restart procedure to the raw invocation topology instead of the learned multimodal propagation graph.

Topo-MFP achieves the highest mean Hit@1 and MAP@3 values among the evaluated RCA baselines on both datasets. On AIOps2022, Topo-MFP improves Hit@1 from 0.667 ± 0.016 obtained by DiagFusion, the strongest evaluated existing-method baseline, to 0.872 ± 0.010. Compared with the added Topology-RWR baseline, Topo-MFP improves Hit@1 from 0.724 ± 0.015 to 0.872 ± 0.010. On TT, Topo-MFP improves Hit@1 from 0.784 ± 0.014 obtained by MicroHECL, the strongest evaluated existing-method RCA baseline on this dataset, to 0.901 ± 0.009, and improves over Topology-RWR from 0.824 ± 0.014 to 0.901 ± 0.009.

The comparison with Topology-RWR is particularly important because Topology-RWR uses the same candidate set, restart vector, restart probability, and reverse random walk mechanism as Topo-MFP, but performs ranking on the raw invocation topology. The consistent improvement of Topo-MFP over Topology-RWR indicates that the learned multimodal propagation graph provides additional diagnostic value beyond raw topology and reverse random walk alone.

The Random reference rows in Tables 18 and 19, computed from the test-split candidate-set sizes, show that Hit@5 can be relatively high when candidate sets are moderate in size. Therefore, Hit@5 should be interpreted cautiously. In the following analysis, Hit@1 and MAP@3 are emphasized because they better reflect whether the true root-cause service is ranked near the top of the candidate list.

6.5 Effect of log semantic encoders

Tables 20 and 21 compare different log semantic encoders in the RCA task under the default experimental setting. The

Table 14 Fault detection results on AIOps2022 over five random seeds

Method	Precision	Recall	F1-score	AUC
UAC-AD	0.904 ± 0.006	0.915 ± 0.007	0.909 ± 0.006	0.922 ± 0.005
USAD	0.895 ± 0.008	0.871 ± 0.009	0.883 ± 0.008	0.913 ± 0.006
TraceVAE	0.913 ± 0.005	0.956 ± 0.006	0.934 ± 0.005	0.932 ± 0.005
Topo-MFP	0.937 ± 0.005	0.961 ± 0.004	0.949 ± 0.004	0.956 ± 0.003

Results are reported as mean ± standard deviation over five random seeds

Table 15 Fault detection results on TT over five random seeds

Method	Precision	Recall	F1-score	AUC
UAC-AD	0.865 ± 0.007	0.882 ± 0.008	0.873 ± 0.007	0.930 ± 0.005
USAD	0.793 ± 0.010	0.821 ± 0.011	0.807 ± 0.010	0.927 ± 0.006
TraceVAE	0.929 ± 0.005	0.951 ± 0.005	0.940 ± 0.005	0.933 ± 0.004
Topo-MFP	0.947 ± 0.005	0.955 ± 0.005	0.951 ± 0.004	0.963 ± 0.004

Results are reported as mean ± standard deviation over five random seeds

Table 16 Modality ablation on AIOps2022

Variant	Precision	Recall	F1-score	AUC
w/o log semantics	0.876	0.902	0.889	0.905
w/o traces	0.901	0.927	0.914	0.921
w/o metrics	0.908	0.924	0.916	0.927
Topo-MFP	0.937	0.961	0.949	0.956

Table 17 Module ablation on TT

Variant	Precision	Recall	F1-score	AUC
w/o SGAM	0.914	0.941	0.927	0.936
w/o TGAM	0.905	0.929	0.917	0.924
w/o CGAM	0.927	0.945	0.936	0.947
Topo-MFP	0.947	0.955	0.951	0.963

rest of the RCA framework is kept unchanged, and only the log template representation method is replaced. This auxiliary study is used to analyze the effect of semantic log representation, while the repeated-run main RCA comparisons are reported in Tables 18 and 19.

SBERT achieves the best performance on both datasets. On AIOps2022, SBERT improves Hit@1 from 0.741 with TF-IDF and 0.803 with BERT to 0.872. On TT, SBERT improves Hit@1 from 0.781 with TF-IDF and 0.842 with BERT to 0.901. TF-IDF only captures word frequency statistics and cannot model contextual semantics between short log templates. BERT captures contextual information, but it is not directly optimized for sentence-level similarity. SBERT provides more stable sentence-level embeddings for short structured log templates, which helps distinguish semantically related fault messages and lexically similar but operationally different templates. This observation is consistent with the ablation results showing that log semantic modeling contributes to root-cause localization.

6.6 RCA ablation study

Tables 22 and 23 report the RCA ablation results under the default experimental setting. Three variants are evaluated:

Table 18 Root cause localization results on AIOps2022 over five random seeds

Method	Hit@1	Hit@3	Hit@5	MAP@3
Random reference	0.169	0.497	0.779	0.294
Local-Score	0.584 ± 0.018	0.742 ± 0.017	0.846 ± 0.015	0.674 ± 0.016
Topology-RWR	0.724 ± 0.015	0.851 ± 0.014	0.912 ± 0.012	0.796 ± 0.014
MRCA	0.659 ± 0.017	0.768 ± 0.016	0.825 ± 0.015	0.721 ± 0.015
MicroHECL	0.651 ± 0.018	0.753 ± 0.017	0.839 ± 0.015	0.712 ± 0.016
DiagFusion	0.667 ± 0.016	0.781 ± 0.015	0.812 ± 0.016	0.733 ± 0.015
Topo-MFP	0.872 ± 0.010	0.917 ± 0.008	0.963 ± 0.006	0.891 ± 0.009

Results except Random reference are reported as mean ± standard deviation over five random seeds. Random reference is the expected value computed from the test-split candidate-set sizes and is not averaged over training seeds. Paired significance tests against Topology-RWR are reported in Table 13

Table 19 Root cause localization results on TT over five random seeds

Method	Hit@1	Hit@3	Hit@5	MAP@3
Random reference	0.147	0.434	0.702	0.256
Local-Score	0.692 ± 0.017	0.821 ± 0.015	0.889 ± 0.013	0.766 ± 0.015
Topology-RWR	0.824 ± 0.014	0.903 ± 0.012	0.936 ± 0.010	0.866 ± 0.012
MRCA	0.773 ± 0.015	0.824 ± 0.014	0.869 ± 0.013	0.811 ± 0.013
MicroHECL	0.784 ± 0.014	0.817 ± 0.014	0.851 ± 0.013	0.805 ± 0.013
DiagFusion	0.761 ± 0.015	0.832 ± 0.013	0.887 ± 0.012	0.817 ± 0.012
Topo-MFP	0.901 ± 0.009	0.945 ± 0.007	0.961 ± 0.006	0.921 ± 0.008

Results except Random reference are reported as mean ± standard deviation over five random seeds. Random reference is the expected value computed from the test-split candidate-set sizes and is not averaged over training seeds. Paired significance tests against Topology-RWR are reported in Table 13

Table 20 Effect of log semantic encoders on AIOps2022

Encoder	Hit@1	Hit@3	Hit@5	MAP@3
TF-IDF	0.741	0.812	0.857	0.776
BERT	0.803	0.865	0.912	0.834
SBERT	0.872	0.917	0.963	0.891

Table 21 Effect of log semantic encoders on TT

Encoder	Hit@1	Hit@3	Hit@5	MAP@3
TF-IDF	0.781	0.842	0.886	0.812
BERT	0.842	0.903	0.938	0.875
SBERT	0.901	0.945	0.961	0.921

removing log semantic modeling, removing topology-aware regularization, and removing contrastive consistency fusion. These ablation studies are used to analyze the contribution of key modules in Topo-MFP, while the repeated-run main RCA comparisons are reported in Tables 18 and 19.

The complete model obtains the best results on both datasets. Removing log semantics reduces Hit@1 from 0.872 to 0.813 on AIOps2022 and from 0.901 to 0.841 on TT. This indicates that semantic logs help distinguish root-cause services from downstream affected services, especially when multiple services exhibit propagated symptoms.

Removing topology-aware regularization also degrades RCA performance. This result shows that topology is useful not only as an inference graph for random walk, but also as a structural prior during propagation graph learning. Without this regularization, the learned propagation graph may include more topology-distant noisy edges, which weakens symptom-to-source ranking.

Removing contrastive consistency fusion reduces multimodal agreement and leads to lower Hit@1 and MAP@3. This suggests that consistent evidence across metrics, logs, and traces is useful for learning reliable diagnosis-oriented propagation patterns.

Together with the Topology-RWR baseline in Tables 18 and 19, these ablation results clarify the role of topology in Topo-MFP. Topology-RWR uses the raw invocation topology directly for reverse random walk, whereas the full Topo-MFP learns a multimodal propagation graph guided by topology-aware regularization. The improvement of Topo-MFP over both Topology-RWR and the w/o topology regularization variant indicates that raw topology provides useful structural guidance, while learned multimodal propagation evidence further improves root-cause ranking.

Table 22 RCA ablation results on AIOps2022

Variant	Hit@1	Hit@3	Hit@5	MAP@3
w/o log semantics	0.813	0.862	0.891	0.851
w/o topology regularization	0.837	0.883	0.912	0.865
w/o contrastive consistency fusion	0.821	0.887	0.904	0.871
Topo-MFP	0.872	0.917	0.963	0.891

6.7 Parameter sensitivity

Tables 24 and 25 report the sensitivity results of historical window size and log semantic dimension on the TT dataset under the default experimental setting. These experiments analyze how key hyperparameters affect RCA performance.

The performance improves as the historical window size increases from 5 to 20. A short historical window may not capture sufficient temporal propagation evidence, while an excessively long window may introduce redundant or stale observations. The best performance is achieved around a window size of 20. Increasing the log semantic dimension also improves performance, but the gain becomes marginal after 256 dimensions. Therefore, 256 is used as the default log semantic dimension for RCA to balance performance and representation cost.

Table 26 further reports the sensitivity to λ_2, the coefficient of topology-aware sparse regularization. This

Table 23 RCA ablation results on TT

Variant	Hit@1	Hit@3	Hit@5	MAP@3
w/o log semantics	0.841	0.894	0.905	0.877
w/o topology regularization	0.879	0.907	0.947	0.892
w/o contrastive consistency fusion	0.871	0.921	0.933	0.897
Topo-MFP	0.901	0.945	0.961	0.921

parameter controls the strength of the topology-aware structural prior in propagation graph learning.

When $\lambda_2 = 0$, the topology-aware sparsity prior is removed and the performance decreases, which is consistent with the ablation result of removing topology regularization. Moderate values improve root cause localization by discouraging topology-distant noisy edges while preserving data-supported implicit dependencies. The best performance is obtained at $\lambda_2 = 0.01$. When λ_2 becomes too large, the learned propagation graph may be over-constrained by the raw topology, reducing the contribution of multimodal temporal evidence. This trend further supports the design choice of using topology as a soft prior rather than as a fixed causal graph.

7 Summary of findings

The experimental results lead to four main observations. First, Topo-MFP shows a consistent improvement trend in service-level fault detection on both datasets, although the detection margin over the strongest baseline is moderate. Therefore, repeated-run reporting is important for assessing the stability of the observed improvement. Second, RCA improvements are more substantial, especially in Hit@1 and MAP@3, indicating that symptom-to-source ranking over a learned propagation graph is useful for distinguishing root-cause services from downstream affected services. Third, the expected random-ranking reference, Local-Score, and Topology-RWR help clarify the source of RCA improvement. The random-ranking reference shows the influence of candidate-set size, Local-Score shows the limitation of using only local abnormal intensity, and Topology-RWR shows the contribution of raw topology and reverse random walk. The improvement of Topo-MFP over Topology-RWR further indicates the value of learning multimodal propagation evidence. Fourth, ablation and sensitivity results show that semantic log modeling, topology-aware regularization, and cross-modal consistency all contribute to the final performance.

Overall, the results support the effectiveness of Topo-MFP under the adopted service-level evaluation protocol. The learned propagation graph should be interpreted as a diagnosis-oriented ranking representation rather than as a verified causal graph.

Table 24 Sensitivity to historical window size on TT

Window size	Hit@1	Hit@3	Hit@5	MAP@3
5	0.812	0.863	0.878	0.858
10	0.888	0.935	0.956	0.910
15	0.896	0.941	0.959	0.916
20	0.901	0.945	0.961	0.921
25	0.898	0.942	0.959	0.918
30	0.892	0.937	0.956	0.913

Table 25 Sensitivity to log semantic dimension on TT

Semantic dimension	Hit@1	Hit@3	Hit@5	MAP@3
64	0.882	0.930	0.952	0.904
128	0.894	0.938	0.957	0.913
256	0.901	0.945	0.961	0.921
512	0.902	0.946	0.962	0.922
768	0.903	0.947	0.963	0.923

Table 26 Sensitivity to λ_2 for topology-aware sparse regularization on TT

λ_2	Hit@1	Hit@3	Hit@5	MAP@3
0	0.879	0.907	0.947	0.892
0.001	0.891	0.928	0.954	0.909
0.005	0.897	0.939	0.958	0.917
0.01	0.901	0.945	0.961	0.921
0.05	0.894	0.936	0.956	0.914
0.10	0.887	0.929	0.952	0.907

8 Discussion

8.1 Effectiveness of propagation graph learning

Topo-MFP learns a diagnosis-oriented propagation dependency graph from temporal precedence, topology priors, and multimodal abnormal evidence. The graph is designed to support root cause ranking by modeling how abnormal symptoms may spread across service dependencies. This design is useful in microservice systems because root-cause services and affected services may show different abnormal patterns across logs, metrics, and traces.

The learned propagation graph should be interpreted as a diagnostic representation rather than as proof of intervention-level causality. Topo-MFP does not claim that the learned propagation graph recovers ground-truth causal or propagation edges, because the public datasets used in this paper do not provide edge-level propagation annotations.

In real microservice systems, retries, queues, asynchronous calls, shared resources, and feedback loops may affect symptom propagation. Therefore, the experimental results demonstrate the usefulness of the learned graph for RCA ranking, rather than the edge-level correctness of the recovered propagation structure.

8.2 Understanding the RCA improvement

Topo-MFP shows relatively large RCA improvement over the evaluated baselines, especially on AIOps2022. This improvement may be attributed to three factors. First, Topo-MFP explicitly performs symptom-to-source ranking by reverse random walk over the learned propagation graph, while several evaluated baselines rely more heavily on local abnormal scores, partial modalities, or rule-based propagation paths. Second, semantic log modeling helps distinguish root causes from downstream symptoms, because root services may emit more informative error logs before downstream services show generic timeout or failure symptoms. Third, topology-aware sparse regularization biases the learned propagation graph toward topology-adjacent service pairs while still allowing implicit dependencies when multimodal evidence is strong.

The RCA improvement should be interpreted within the adopted service-level evaluation protocol. The average number of services involved in each diagnosis window is moderate, and the evaluated baselines are selected because they can be consistently adapted to the same input granularity and candidate service set. Future work will include larger-scale service graphs, additional heuristic ranking baselines, and more recent RCA methods under a unified protocol.

8.3 Experimental scope

The experiments in this paper are designed to evaluate whether Topo-MFP can improve service-level fault detection and RCA under a unified protocol on two available multimodal microservice datasets. The study does not fully characterize all possible deployment conditions. In particular, it does not include large-scale industrial service graphs with hundreds or thousands of services, repeated-run analysis for all auxiliary experiments, or edge-level ground-truth propagation paths. Therefore, the reported results mainly support the effectiveness of the proposed framework in the evaluated datasets and should be further validated in larger and more diverse production environments.

8.4 Complexity analysis

Let n be the number of services, m be the number of log templates, $|E_s|$ be the number of service dependency edges, $|E_l|$ be the number of log template graph edges, w be the historical window length, and d be the hidden dimension.

The complexity of semantic-transition log template graph attention is approximately:

$$O(|E_l| d).$$

The spatial service graph attention module operates on the trace-induced service dependency graph. Its complexity over a historical window is approximately:

$$O(w |E_s| d).$$

The temporal attention module aggregates service representations across the historical window. Its complexity is approximately:

$$O(nw^2 d).$$

For propagation graph learning in the RCA module, Topo-MFP computes propagation scores over service pairs within the diagnosis-window candidate set. In the worst case, if all service pairs are considered, the complexity is approximately:

$$O(wn^2 d).$$

The topology-aware sparsity prior reduces the effective density of the learned propagation graph by assigning smaller penalties to topology-adjacent service pairs and larger penalties to topology-distant service pairs. However, this regularization term does not change the worst-case computational complexity by itself, because propagation scores still need to be computed before sparsity is imposed. Therefore, without explicit top-k or threshold-based pruning, the worst-case complexity of propagation graph learning remains:

$$O(wn^2 d).$$

In the evaluated datasets, the number of services involved in each diagnosis window is moderate, as reported in the candidate-set size statistics. Therefore, all-pair propagation scoring within the diagnosis-window candidate set remains computationally feasible. In larger production systems, explicit sparse pruning, such as retaining topology-adjacent edges and the top-k topology-distant edges for each service, can be introduced to reduce practical message-passing cost. If such pruning is applied, and $|E_{prop}|$ denotes the retained propagation edge set, the practical message-passing complexity becomes:

$$O(w |E_{prop}| d).$$

When the retained propagation edge set is close to the observed service dependency graph, this practical complexity further approaches:

$O(w \mid E_s \mid d)$.

Thus, the current implementation is suitable for moderate-scale diagnosis-window candidate sets, while explicit pruning is a useful extension for scaling Topo-MFP to larger microservice systems.

8.5 Limitations and future mechanism validation

The current experiments validate the end-to-end diagnosis performance and module-level contributions. However, direct edge-level validation of the learned propagation graph remains difficult because public datasets usually do not provide ground-truth propagation edges. Future work will further evaluate propagation graph quality through expert inspection, synthetic controlled faults, edge perturbation analysis, and comparison with raw topology or correlation graphs under a unified protocol.

9 Threats to validity

Several threats to validity remain.

First, the experiments are conducted on AIOps2022 and TrainTicket-based TT datasets. Although these datasets contain multimodal observability data, they may not cover all failure patterns, deployment scales, traffic distributions, and operational conditions in industrial production systems.

Second, the evaluated baselines are selected because they can be adapted to the adopted service-level protocol with available inputs. Some recent RCA methods, such as CHASE, RUN, DGERCL, MicroEGRCL, and Grace, are not included in the main empirical comparison due to differences in input assumptions, graph construction requirements, output granularity, or diagnosis targets.

Third, although the main experimental results are reported over five random seeds, some ablation and sensitivity analyses are conducted under the default setting due to computational cost. Therefore, small differences in secondary experiments should be interpreted cautiously.

Fourth, operation-level labels are mapped to service-level labels for unified evaluation. This mapping makes different methods comparable at the service level, but it may hide operation-level differences and introduce adaptation bias.

Fifth, the learned propagation graph is used for diagnosis-oriented ranking. Without intervention data or ground-truth propagation edges, it should be interpreted as a diagnostic representation rather than as a formally identifiable propagation dependency graph.

Sixth, this paper does not claim production deployment readiness. Any prototype-level runtime observation should be interpreted as controlled-setting evidence only. Long-term production deployment, large-scale service expansion, high ingestion pressure, concurrent diagnosis requests, cold-start model loading, and online model updating require further validation.

10 Reproducibility

To improve reproducibility, the implementation package is prepared to include preprocessing scripts, model configuration files, and evaluation scripts where permitted by dataset policies. The package includes scripts for service-level label mapping, data splitting, log parsing, feature normalization, model training, and metric computation.

For datasets that cannot be redistributed due to usage restrictions, the preprocessing scripts will be provided to reproduce experiments from the official dataset sources. The configuration files will record the main hyperparameters used in the experiments, including window size, hidden dimension, temperature, restart probability, dropout, learning rate, batch size, maximum epochs, and early stopping patience.

The fixed random seeds used in the repeated-run evaluation are recorded in the configuration files. The released scripts will support repeated training, variance analysis, and reproducible metric computation where permitted by dataset policies.

11 Conclusion

This paper proposed Topo-MFP, a topology-aware multimodal propagation learning framework for fault detection and root cause localization in microservice systems. The framework was designed to address the difficulty of diagnosing failures that propagate across service dependencies and manifest differently in metrics, logs, and traces. Topo-MFP combines semantic-transition log graph learning, topology-aware service representation learning, diagnosis-oriented multimodal propagation graph construction, and symptom-to-source reverse random walk with restart.

For fault detection, Topo-MFP constructs a semantic-transition log template graph and combines it with dynamic service dependency graphs to learn service-level abnormal representations. For root cause localization, it learns modality-specific propagation graphs from multimodal temporal observations, fuses them through cross-modal consistency, and applies a topology-aware soft sparsity prior to distinguish topology-adjacent and topology-distant service pairs. The fused propagation graph is then used for reverse random-walk ranking, which traces observed symptoms back to likely upstream root-cause services.

Experiments on AIOps2022 and a TrainTicket-based TT dataset showed that Topo-MFP achieved competitive and stable performance under the adopted service-level evaluation protocol. Over five random seeds, Topo-MFP obtained F1-scores of 0.949 ± 0.004 and 0.951 ± 0.004 for fault detection, and Hit@1 scores of 0.872 ± 0.010 and 0.901 ± 0.009 for root cause localization on AIOps2022 and TT, respectively. The comparison with the Random reference, Local-Score, and Topology-RWR further showed that the observed RCA improvement does not come solely from candidate-set size, local anomaly intensity, reverse random walk, or raw invocation topology, but is associated with the learned multimodal propagation graph. Ablation studies confirmed the effectiveness of semantic log modeling, cross-modal consistency, and topology-aware regularization, while sensitivity analysis on the topology regularization coefficient supported the use of service topology as a soft prior rather than a fixed propagation structure.

The learned propagation graph in Topo-MFP should be interpreted as a diagnosis-oriented ranking representation rather than as a verified causal graph. Although the experimental results support the effectiveness of the proposed framework on the evaluated datasets, several limitations remain. First, the evaluation is conducted on two benchmark-style datasets and does not yet cover very large industrial microservice systems with hundreds or thousands of services. Second, edge-level ground-truth propagation paths are not available, which limits direct validation of the learned propagation graph. Third, repeated-run evaluation is reported for the main comparisons, while some auxiliary studies are conducted under the default setting due to computational cost. Future work will focus on larger-scale industrial validation, stronger generalization to unseen fault types, additional RCA baselines under unified protocols, and direct validation of learned propagation graphs using controlled fault injection.

Author contributions Z.P. conceived the research idea, designed the Topo-MFP method, implemented the model, performed data preprocessing and experiments, analyzed the experimental results, and wrote the original draft. W.L. supervised the study, provided methodological guidance, and revised the manuscript critically. J.W. contributed to research discussion, experimental design, and result interpretation. H.H. assisted with dataset processing, baseline adaptation, and experimental validation. L.Y. contributed to prototype system implementation, visualization, and manuscript preparation. All authors reviewed and approved the final manuscript.

Funding No funding was received for this study.

Data availability No datasets were generated or analysed during the current study.

Declarations

Ethics approval Not applicable.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Audibert J, Michiardi P, Guyard F, Marti S, Zuluaga MA (2020) USAD: unsupervised anomaly detection on multivariate time series. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 3395–3404

Baškarada S, Nguyen V, Koronios A (2020) Architecting microservices: practical opportunities and challenges. J Comput Inf Syst. https://doi.org/10.1080/08874417.2018.1520056

Bogatinovski J, Nedelkoski S, Cardoso J, Kao O (2020) Self-supervised anomaly detection from distributed traces. In: Proceedings of the 2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing, pp 342–347

Chen R, Ren J, Wang L et al (2022) MicroEGRCL: An edge-attention-based graph neural network approach for root cause localization in microservice systems. In: Proceedings of the International Conference on Service-Oriented Computing, pp 264–272

Cheng H, Li Q, Liu B et al (2024) DGERCL: a dynamic graph embedding approach for root cause localization in microservice systems. IEEE Trans Serv Comput. https://doi.org/10.1109/TSC.2024.3437742

Cho K, Van Merriënboer B, Gulcehre C et al (2014) Learning phrase representations using RNN encoder-decoder for statistical machine translation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, pp 1724–1734

Cinque M et al (2016) Characterizing direct monitoring techniques in software systems. IEEE Trans Reliab 65(4):1665–1681

Dragoni N, Giallorenzo S, Lluch Lafuente A et al (2017) Microservices: yesterday, today, and tomorrow. Present and ulterior software engineering. Springer

Du M, Li F, Zheng G, Srikumar V (2017) DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp 1285–1298

Di Francesco P, Malavolta I, Lago P (2017) Research on architecting microservices: Trends, focus, and potential for industrial adoption. In: Proceedings of the 2017 IEEE International Conference on Software Architecture, pp 21–30

Gan Y, Zhang Y, Hu K et al (2019) Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, pp 19–33

He P, Zhu J, Zheng Z, Lyu MR (2017) Drain: an online log parsing approach with fixed depth tree. In: Proceedings of the 2017 IEEE International Conference on Web Services, pp 33–40

Jia T, Yang L, Chen P et al (2017) LogSed: Anomaly diagnosis through mining time-weighted control flow graph in logs. In: Proceedings of the 2017 IEEE 10th International Conference on Cloud Computing, pp 447–455

Lewis J, Fowler M (2014) Microservices: a definition of this new architectural term. https://martinfowler.com/articles/microservices.html

Lin CM, Chang C, Wang WY et al (2024) Root cause analysis in microservice using neural Granger causal discovery. Proc AAAI Conf Artif Intell 38:206–213

Liu H, Huang X, Jia M et al (2024) UAC-AD: unsupervised adversarial contrastive learning for anomaly detection on multimodal data in microservice systems. IEEE Trans Serv Comput 17(6):3887–3900

Liu P, Xu H, Ouyang Q et al (2020) Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks. In: Proceedings of the 2020 IEEE 31st International Symposium on Software Reliability Engineering, pp 48–58

Liu D, He C, Peng X et al (2021) MicroHECL: High-efficient root cause localization in large-scale microservice systems. In: Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice, pp 338–347

Meng L, Ji F, Sun Y et al (2021) Detecting anomalies in microservices with execution trace comparison. Future Gener Comput Syst 116:291–301

Meng W, Liu Y, Zhu Y et al (2019) LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, pp 4739–4745

Nandi A, Mandal A, Atreja S et al (2016 Anomaly detection using program control flow graph mining from execution logs. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 215–224

Nedelkoski S, Cardoso J, Kao O (2019) Anomaly detection and classification using distributed tracing and deep learning. In: Proceedings of the 2019 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, pp 241–250

Oord van den A, Li Y, Vinyals O (2018) Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748

Paszke A, Gross S, Massa F et al (2019) PyTorch: an imperative style, high-performance deep learning library. Adv Neural Inf Proces Syst 32

Reimers N, Gurevych I (2019) Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp 3982–3992

Ren R, Wang Y, Liu F et al (2023) Grace: Interpretable root cause analysis by graph convolutional network for microservices. In: Proceedings of the 2023 IEEE/ACM 31st International Symposium on Quality of Service, pp 1–4

Sigelman BH, Barroso LA, Burrows M et al (2010) Dapper, a large-scale distributed systems tracing infrastructure. Google

Tong H, Faloutsos C, Pan JY (2006) Fast random walk with restart and its applications. In: Proceedings of the Sixth International Conference on Data Mining, pp 613–622

Vaswani A, Shazeer N, Parmar N et al (2017) Attention is all you need. Adv Neural Inf Process Syst 5998–6008

Wang Y, Zhu Z, Fu Q et al (2024) MRCA: Metric-level root cause analysis for microservices via multi-modal data. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp 1057–1068

White G, Diuwe J, Fonseca E et al (2021) MMRCA: Multimodal root cause analysis. In: Proceedings of the International Conference on Service-Oriented Computing, pp 177–189

Wu L, Tordsson J, Elmroth E et al (2020) MicroRCA: Root cause localization of performance issues in microservices. In: Proceedings of the IEEE/IFIP Network Operations and Management Symposium

Xie Z, Xu H, Chen W et al (2023) Unsupervised anomaly detection on microservice traces through graph VAE. In: Proceedings of the ACM Web Conference 2023, pp 2874–2884

Zhang S, Jin P, Lin Z et al (2023) Robust failure diagnosis of microservice system through multimodal data. IEEE Trans Serv Comput 16(6):3851–3864

Zhang X, Xu Y, Lin Q et al (2019) Robust log-based anomaly detection on unstable log data. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp 807–817

Zhao Z, Wang Z, Zhang T et al (2024) CHASE: a causal hypergraph based framework for root cause analysis in multimodal microservice systems. arXiv preprint arXiv:2406.19711

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.